

Active IR

Line-Following Robot

From 940 nm photons to autonomous path-following

ZEKE MOHAMMED
KIRSTEN DAVIS

What we'll cover



01

History of MCU Programming

Binary · Assembly · Embedded C · Arduino

KIRSTEN



02

IR Sensors — Physics & Principles

EM spectrum · photodiode · thresholding

KIRSTEN + ZEKE



03

PCB & Design

Layers · traces · the HW-130 dissected

ZEKE



04

Results & Future Work

Track performance · MPU-6050 as v3

ZEKE

It started with zeros and ones

Machine code → Assembly — one-to-one with the CPU's wiring

1947

Assembly language

Kathleen Booth writes the first assembler, translating human-readable mnemonics like MOV and JMP into the raw binary opcodes a CPU actually executes.

ONE INSTRUCTION, THREE VIEWS

BINARY	HEX	ASSEMBLY
10110000 01100001	B0 61	MOV AL, 0x61 // Load 'a' into register
00000100 00000001	04 01	ADD AL, 1 // increment by 1
11101011 11111110	EB FE	JMP \$ // jump to self (halt)

Assembly meets the microcontroller

The 1970s — CPU, RAM, ROM, and I/O all on a single chip you could program directly

For two decades, **assembly** was the only practical way to program these chips — every byte of ROM was precious.



WHY IT MATTERS TODAY: Modern Arduino code still compiles down to AVR assembly — it's C++ on the surface, but the silicon sees MOV, LDI, and JMP.

C takes over — 1972

Portable, structured, and fast enough for bare-metal work

1972

C

Dennis Ritchie · Bell Labs

- Direct memory access via pointers
- Compiles to efficient machine code
- Used to write the UNIX kernel

1983

C++

Bjarne Stroustrup

- Classes and object-oriented code
- Zero-overhead abstraction principle
- The Arduino language is C++

1991

Python

Guido van Rossum

- Readable syntax — "code as thoughts"
- MicroPython / CircuitPython bring it to MCUs
- Slower; not for hard real-time

TAKEAWAY *C gave us a language close enough to the hardware to replace assembly for most work — but readable enough for humans to maintain.*

Where we are now

Programming microcontrollers in 2026

2005

Arduino C++

Wiring-based abstraction over AVR-GCC. Massive library ecosystem. What this robot runs.

WE USE THIS

2014

MicroPython

Python-like syntax running a bytecode interpreter on-chip. Beginner-friendly, interactive REPL.

2018

Rust Embedded

Memory safety without garbage collection — compile-time guarantees for bare-metal code.

ASM

Still Used

Boot code, interrupt service routines, tight DSP loops. Every compiler still emits assembly.

FOUNDATION

THE PATTERN: *Higher abstractions keep arriving — but they all ultimately produce the same kind of instructions a 1947 assembler would recognize.*

What is an IR sensor?

A device that detects infrared electromagnetic radiation — wavelengths longer than visible light

RANGE

0.7 – 14 μm

micrometers — just past red, invisible to the human eye

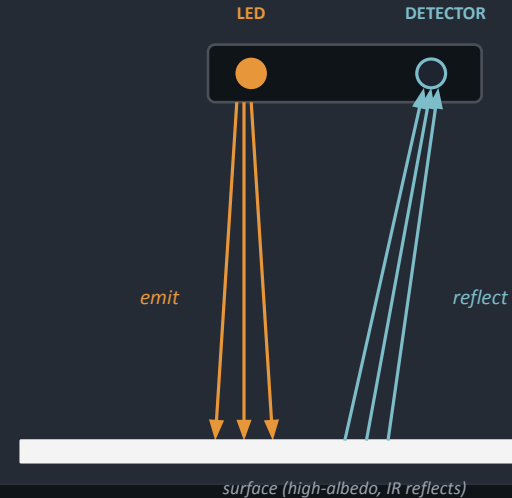
ACTIVE *(what we use)*

Emits its own IR and measures what bounces back. The sensor is both the flashlight and the eye.

PASSIVE

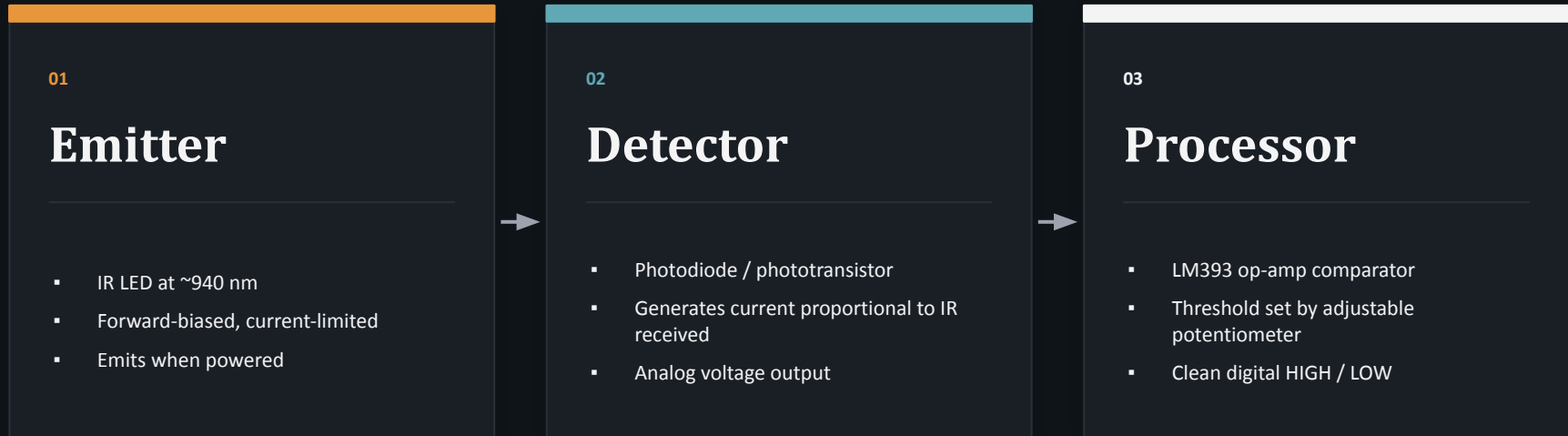
Detects IR that objects already emit (all objects above 0 K radiate). Used in thermal cameras and motion-activated lights.

HOW AN ACTIVE IR SENSOR SEES



Inside an IR sensor module

Three functional blocks: emit · detect · decide



Analog world → Amplify → Digitize → Microcontroller reads a clean 1 or 0

Applications of IR sensing

The same physics, very different uses



Object Detection

Line followers, obstacle avoidance, proximity sensing, break-beam counters, touchless faucets.



Temperature

Non-contact thermometers and thermal cameras read the IR an object emits from its own heat.



Chemical Analysis

IR spectroscopy: molecules absorb specific IR wavelengths based on their bonds. Used to identify compounds.

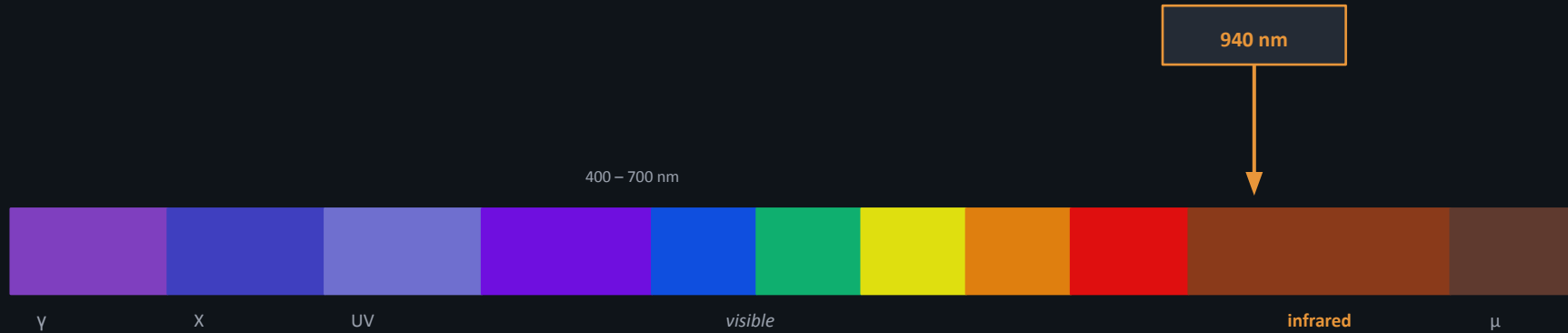


Communication

TV remote controls, IrDA data transfer, night-vision illumination — invisible, cheap, low-interference.

Where 940 nm lives on the spectrum

Why this specific wavelength? Cheap silicon, invisible to us, and outside the peak of indoor lighting.



Invisible to humans

No distracting glow on the robot — the IR LED is bright but we can't see it.

Cheap silicon peak

Standard silicon photodiodes peak near 900-950 nm. Commodity detectors.

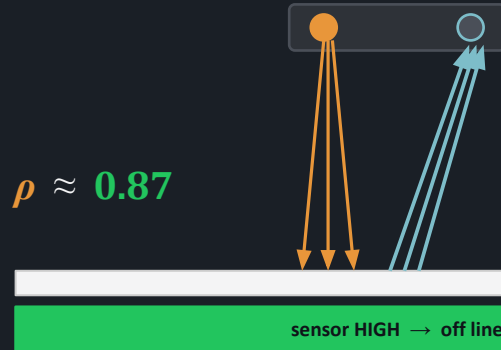
Low ambient noise

Indoor fluorescents and LEDs emit little 940 nm — the signal-to-noise ratio stays high.

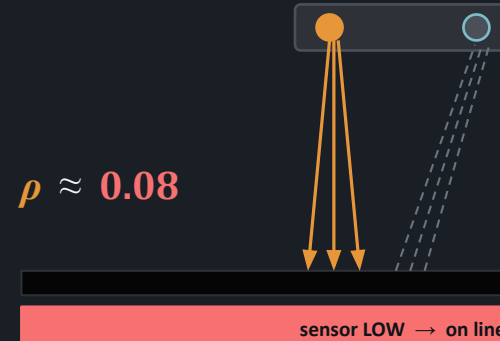
Why black tape works: photon-matter interaction

The sensor doesn't "see color" — it measures reflected IR flux at one wavelength

WHITE SURFACE · HIGH ALBEDO



BLACK TAPE · CARBON PIGMENT



THE PHYSICS

$$\alpha + \rho + \tau = 1$$

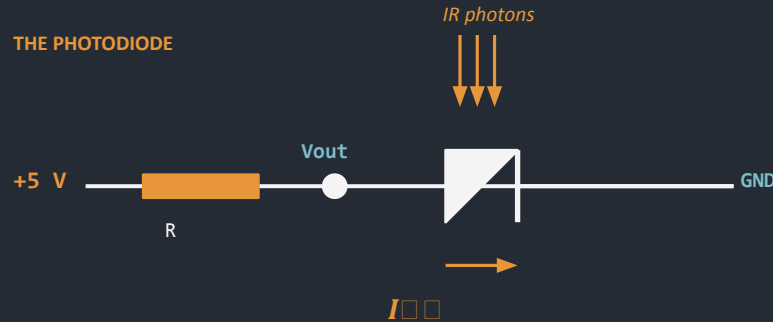
absorptance + reflectance + transmittance

For opaque tape, $\tau = 0$, so $\alpha = 1 - \rho$. Difference in returned photons between white paper and black tape is about **an order of magnitude** — plenty of margin for the comparator to flip reliably. (Kirchhoff's law: good absorber = good emitter.)

Detection: photons become current

The photodiode turns IR light into a measurable signal via the photoelectric effect

THE PHOTODIODE



$$I = \eta \cdot q \cdot \Phi / h\nu$$

η quantum efficiency · Φ incident radiant flux · $h\nu$ photon energy

PHOTOELECTRIC EFFECT

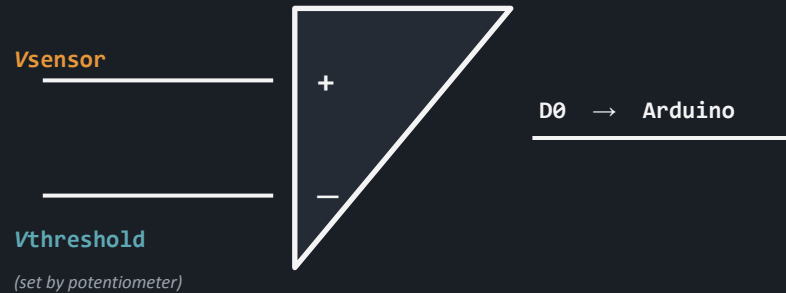
Incident photons promote electrons across the silicon band gap.

- Photon energy must exceed the band gap (~1.1 eV for Si). At 940 nm, $h\nu \approx 1.32$ eV — comfortably above threshold.
- Each absorbed photon liberates one electron–hole pair; the reverse-biased junction sweeps them apart, producing a current.
- More IR hitting the junction = more current = higher voltage across R.

From analog voltage to clean digital signal

The LM393 comparator does a one-bit analog-to-digital conversion, set by a potentiometer

LM393 — AS A COMPARATOR



```
if Vsensor > Vthreshold → D0 = HIGH  
else → D0 = LOW
```

ANALOG IN → DIGITAL OUT

analog



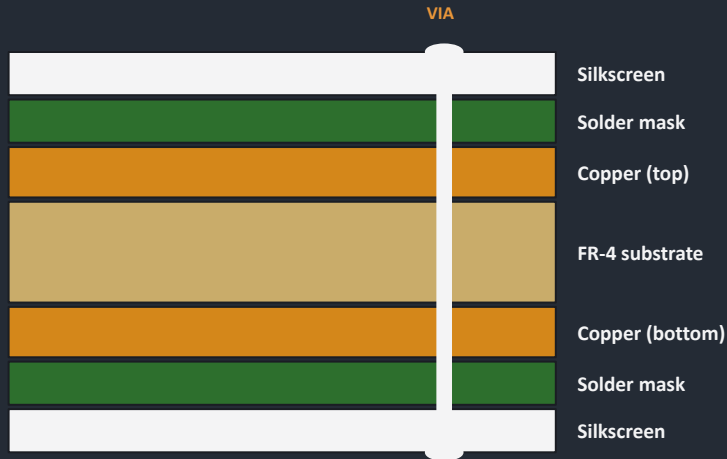
digital



What's actually inside a PCB

A printed circuit board is a sandwich of copper and fiberglass — traces replace wires

CROSS-SECTION · 2-LAYER PCB



Trace

Thin copper path etched into the board — a "wire" you never had to solder.

Pad

Exposed copper landing where a component leg or pin solders down.

Via

Plated-through hole that passes a signal from one copper layer to another.

Substrate

The insulating core — FR-4: flame-retardant woven fiberglass cloth in epoxy resin.

From idea to fabricated board

The modern design pipeline — tools like KiCad, Altium, and Eagle walk you through each step

01

Schematic

Logical circuit diagram — every component and every net, no physical geometry yet.

`.sch file`

02

Layout

Place footprints on the board outline. Decide size, orientation, and thermal flow.

`component placement`

03

Routing

Draw copper traces between pads. Check design rules: trace width, clearance, impedance.

`.kicad_pcb file`

04

Fabrication

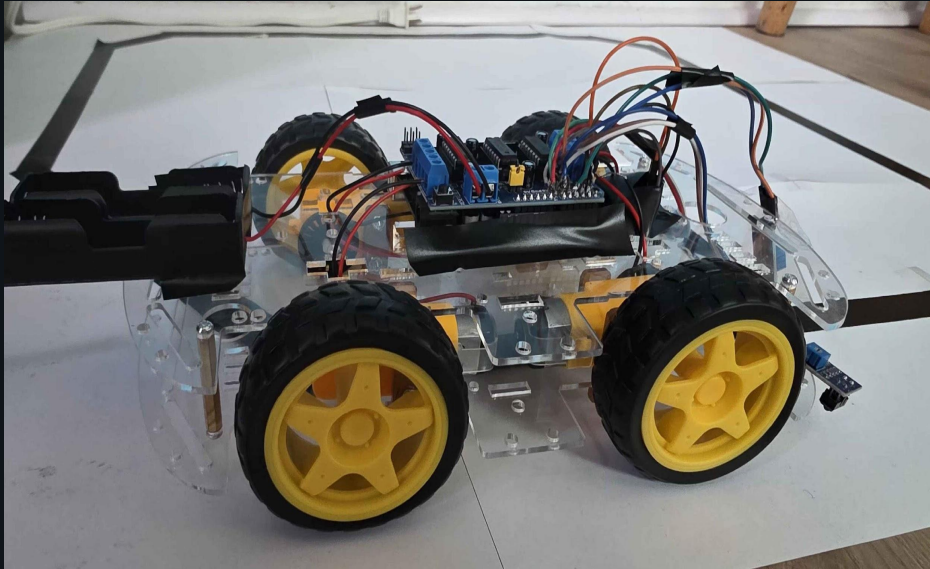
Export Gerber files → send to fab → get bare board → assemble with components.

`physical PCB`

DESIGN RULE CHECK (DRC) catches electrical problems before fabrication: shorts, insufficient clearance, floating nets, unrouted pins.

A real PCB, dissected: the HW-130 motor shield

A clone of the Adafruit Motor Shield V1 — plugs onto the Arduino, adds H-bridge motor control



OUR ROBOT — HW-130 SHIELD VISIBLE ATOP THE ARDUINO UNO

WHAT'S ON THE BOARD

2×

L293D H-bridge IC

Dual quadruple-half-H drivers. Each chip controls 2 motors.

4×

Screw terminals

M1–M4 motor outputs, plus dedicated battery input.

1×

74HC595 shift register

Gives the shield 8 direction lines from only 4 Arduino pins.

16

Male header pins

Plug directly into the Uno's digital and power rails.

∞

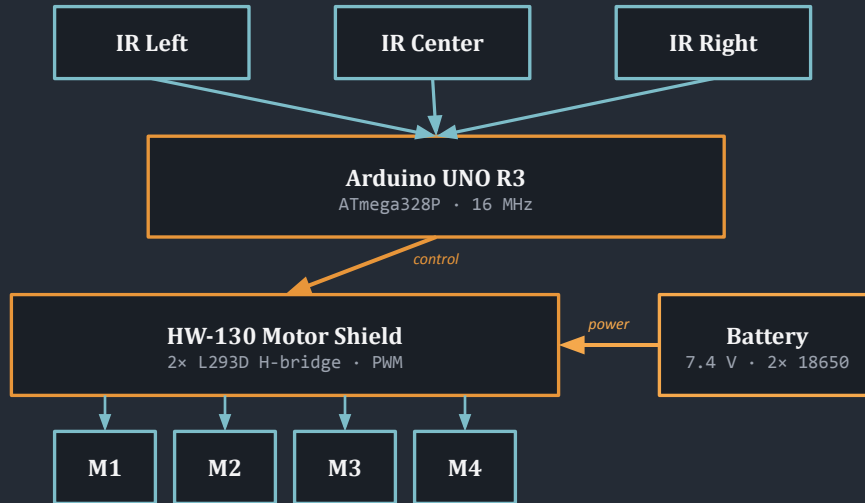
Traces + vias

The copper network that ties every component to every other.

Hardware overview

Four subsystems: sensing, compute, actuation, and power

SYSTEM BLOCK DIAGRAM



Brain

Arduino UNO R3
ATmega328P · 16 MHz · 5 V



Drive

4 × DC gearmotors
L293D dual H-bridge · PWM



Senses

3 × MH-series IR modules
Left · Center · Right



Power

2 × 18650 Li-Ion (series)
~7.4 V nominal

How it decides — and how we got there

Three iterations of tuning before the robot moved cleanly

SENSOR → ACTION LOOKUP

L	C	R	meaning	action
0	1	0	centered	drive straight
1	1	0	drifting right	ease left
0	1	1	drifting left	ease right
1	0	0	line on left	sharp left
0	0	1	line on right	sharp right
0	0	0	lost line	search last heading
1	1	1	intersection	stop

TUNING JOURNEY — WHAT WE TRIED

v1

Two sensors

fail

Tried: Simple binary: "on line / off line"

Result: Overshot every curve — no way to tell HOW far off the line it was.

v2

Step-scan-step

too slow

Tried: Stop · read · pulse forward · stop · repeat

Result: Logic worked, but the robot twitched forward in half-centimeter increments. Unusable.

v3

Continuous differential

works

Tried: 3 sensors + asymmetric wheel speeds on curves

Result: On sharp turns the inner wheels drop to 0 while outer wheels hold speed → smooth arcs.

It works.

Full-loop autonomous line-following on electrical tape tracks

3

sensors

Left · Center · Right — enough to detect drift before losing the line

200

Hz

Decision rate — sensors polled every 5 ms in the main loop

7.4

volts

From two 18650 Li-Ion cells in series — well within the shield's 12 V max

DEMO BEHAVIORS — OBSERVED

- ✓ Completes full closed loop on electrical tape track
- ✓ Handles gradual curves with smooth differential drive
- ✓ Works on both printer paper and hard flooring
- ✓ Recovers from lost line using last known turn direction
- Sharp 90° corners — not yet tested
- Intersection / fork handling — logic written, untested

What we'd do next

Every iteration taught us something the previous one couldn't

LESSONS LEARNED

Verify voltage against spec.

Our first battery pack put 4 cells in series (~14.8 V), exceeding the shield's 12 V maximum. We replaced the shield and switched to 2 × 18650 at 7.4 V.

Sensor count matters more than sensor quality.

2 sensors could only say "on/off line." Adding a center sensor turned binary steering into proportional correction — the biggest single upgrade.

Pin conflicts are invisible until they aren't.

The HW-130 silently uses pins 4, 7, 8, and 12. Sensors on those pins gave nonsensical readings. Moving to analog pins fixed everything.

FUTURE WORK — V3

MPU-6050

6-axis IMU · accelerometer + gyroscope · I²C @ 0x68

- Closed-loop turn control — the gyro tells us exactly 90° for intersections
- Drift correction on long straights before the sensors ever see it
- Inertial dead-reckoning when all three IR sensors lose the line
- Smooth PID control replacing discrete action lookup

THANK YOU — QUESTIONS WELCOME